

Министерство образования Российской Федерации
Пензенский государственный университет
Кафедра «Вычислительная техника»

ОТЧЕТ
по лабораторной работе №8
по курсу «Логика и основы алгоритмизации в
инженерных задачах»
на тему «Обход графа в ширину»

Выполнил:

студенты группы 24ВВВ2

Бауэр А.А.

Гусев Е.В.

Медведев Е.А.

Принял:

к.т.н. доцент Юрова О.В.

к.т.н. Деев М.В.

Пенза 2025

Название

Обход графа в ширину.

Цель работы

Научиться производить обход в графа в ширину.

Лабораторное задание

Задание 1

1. Сгенерируйте (используя генератор случайных чисел) матрицу смежности для неориентированного графа G . Выведите матрицу на экран
2. Для сгенерированного графа осуществите процедуру обхода в ширину, реализованную в соответствии с приведенным выше описанием. При реализации алгоритма в качестве очереди используйте класс **queue** из стандартной библиотеки C++.
3. *Реализуйте процедуру обхода в ширину для графа, представленного списками смежности.

Задание 2

1. *Для матричной формы представления графов реализуйте алгоритм обхода в ширину с использованием очереди, построенной на основе структуры данных «список», самостоятельно созданной.
2. *Оцените время работы двух реализаций алгоритмов обхода в ширину (использующего стандартный класс **queue** и использующего очередь, реализованную самостоятельно) для графов разных порядков.

Описание метода решения задачи

Метод решения основан на алгоритме обхода графа в ширину (BFS) с использованием различных комбинаций структур данных. Граф представляется через матрицу смежности или списки смежности, а очередь организуется с помощью стандартного контейнера STL либо самодельной очереди на основе связного списка.

Алгоритм начинает обход со стартовой вершины, последовательно посещая всех соседей на каждом уровне. Для матричного представления поиск соседей требует просмотра всей строки матрицы, тогда как списки смежности обеспечивают прямой доступ к смежным вершинам. Самодельная очередь реализует классическую структуру FIFO с указателями на начало и конец.

Во 2 задании сравнили скорость обхода 2 методов: queue и CustomQueue. Во всех случаях CustomQueue оказалась быстрее, чем классический вариант. Это происходит за счет того, что в библиотеке учитываются всевозможные случаи работы, что замедляет процесс обхода, рукописный вариант является более “простым” в плане реализации, но при этом менее приспособленным к различным ситуациям.

Листинг

Файл LR8cpp (1 и 2 задание)

```
#include <iostream>
#include <vector>
#include <queue>
#include <list>
#include <ctime>
#include <chrono>
#include <random>
#include <iomanip>
#include <limits>
#include <string>
using namespace std;
using namespace std::chrono;

//Структура элемента очереди
struct QueueNode {
    int data;// Данные - номер вершины
    QueueNode* next;// Указатель на следующий элемент
};

//Самодельная очередь
class CustomQueue {
private:
    QueueNode* front;// Указатель на начало очереди
    QueueNode* rear;// Указатель на конец очереди

public:
    //Конструктор
    CustomQueue() : front(nullptr), rear(nullptr) {}
    //Деструктор для очистки памяти
    ~CustomQueue() {
        while (!empty()) {
            pop();
        }
    }
    //Добавление элемента в конец очереди
    void push(int value) {
        QueueNode* newNode = new QueueNode;
        newNode->data = value;
        newNode->next = nullptr;
        if (rear == nullptr) {
            //Если очередь пуста - новый элемент становится и
            //началом и концом
            front = rear = newNode;
        }
        else {
            //Добавляем элемент в конец очереди
            rear->next = newNode;
            rear = newNode; // Обновляем указатель на конец очереди
        }
    }
    //Удаление элемента из начала очереди
    void pop() {
```

```

        if (empty()) {
            return;
        }
        //Сохраним указатель на удаляемый элемент
        QueueNode* temp = front;
        //Перемещаем начало очереди на следующий элемент
        front = front->next;
        //Если после удаления очередь стала пустой
        if (front == nullptr) {
            rear = nullptr; // Обнуляем указатель на конец
        }
        delete temp; // Освобождаем память удаленного элемента
    }
    //Получение первого элемента очереди без удаления
    int frontValue() {
        if (empty()) {
            return -1; // Возвращаем -1 если очередь пуста
        }
        return front->data;
    }
    //Проверка, пуста ли очередь
    bool empty() {
        return front == nullptr;
    }
    //Получение размера очереди
    size_t size() {
        size_t count = 0;
        QueueNode* current = front;
        while (current != nullptr) {
            count++;
            current = current->next;
        }
        return count;
    }
};

//Структура для узла списка смежности
struct AdjListNode {
    int dest; // Номер вершины
    AdjListNode* next; // Указатель на следующий узел
    //Конструктор
    AdjListNode(int d) : dest(d), next(nullptr) {}
};

//Структура для списка смежности
struct AdjList {
    AdjListNode* head; // Указатель на начало списка смежных вершин
};

//Класс для графа (матрица смежности + списки смежности)
class Graph {
private:
    int V; // Количество вершин
    vector<vector<int>>> adjMatrix; // Матрица смежности
    vector<AdjList> adjLists; // Вектор списков смежности
public:

```

```

//Конструктор графа
Graph(int vertices) : V(vertices) {
    //Инициализация матрицы смежности нулями
    adjMatrix.resize(V, vector<int>(V, 0));
    //Инициализация списков смежности
    adjLists.resize(V);
    for (int i = 0; i < V; i++) {
        adjLists[i].head = nullptr;
    }
}
//Деструктор для очистки памяти
~Graph() {
    //Очистка списков смежности
    for (int i = 0; i < V; i++) {
        AdjListNode* current = adjLists[i].head;
        while (current != nullptr) {
            AdjListNode* temp = current;
            current = current->next;
            delete temp;
        }
    }
}
//Генерация случайного графа
void generateRandomGraph() {
    random_device rd;
    mt19937 gen(rd());
    uniform_int_distribution<> dis(0, 1);
    //Генерация матрицы смежности
    for (int i = 0; i < V; i++) {
        for (int j = i + 1; j < V; j++) {
            int edge = dis(gen); // Случайное решение: есть
ребро или нет
            adjMatrix[i][j] = edge;
            adjMatrix[j][i] = edge;
        }
    }
    //Построение списков смежности на основе матрицы
    buildAdjListsFromMatrix();
}

//Построение списков смежности из матрицы смежности
void buildAdjListsFromMatrix() {
    // Очистка существующих списков
    for (int i = 0; i < V; i++) {
        AdjListNode* current = adjLists[i].head;
        //Удаляем все узлы текущего списка
        while (current != nullptr) {
            AdjListNode* temp = current;
            current = current->next;
            delete temp; // Освобождаем память
        }
        adjLists[i].head = nullptr; // Обнуляем указатель
    }
    //Построение новых списков на основе матрицы смежности
    for (int i = 0; i < V; i++) {
        for (int j = 0; j < V; j++) {

```

```

        if (adjMatrix[i][j] == 1) { // Если есть ребро между
i и j
            addEdgeToList(i, j); // Добавляем в список
смежности
        }
    }
}
//Добавление ребра в список смежности для вершины src
void addEdgeToList(int src, int dest) {
    //Создаем новый узел для вершины назначения
    AdjListNode* newNode = new AdjListNode(dest);
    //Вставляем новый узел в начало списка
    newNode->next = adjLists[src].head;
    adjLists[src].head = newNode;
}
//Вывод матрицы смежности
void printAdjMatrix() {
    cout << "Матрица смежности:" << endl;
    cout << "  ";
    //Вывод заголовка
    for (int i = 0; i < V; i++) {
        cout << setw(3) << i;
    }
    cout << endl;
    //Вывод самой матрицы
    for (int i = 0; i < V; i++) {
        cout << setw(2) << i << " "; // Номер строки
        for (int j = 0; j < V; j++) {
            cout << setw(3) << adjMatrix[i][j]; // Элемент
матрицы
        }
        cout << endl;
    }
    cout << endl;
}
//Вывод списков смежности
void printAdjLists() {
    cout << "Списки смежности:" << endl;
    for (int i = 0; i < V; i++) {
        cout << i << ": "; // Номер вершины
        AdjListNode* current = adjLists[i].head;
        //Проходим по всему списку смежных вершин
        while (current != nullptr) {
            cout << current->dest;
            if (current->next != nullptr) {
                cout << " -> ";
            }
            current = current->next;
        }
        cout << endl;
    }
    cout << endl;
}
}

```

```

//Обход в ширину с использованием матрицы смежности и
стандартной очереди (Задание 1)
vector<int> BFS_Matrix_StdQueue(int start) {
    vector<int> result;// Вектор для хранения порядка обхода
    vector<bool> visited(V, false);// Вектор посещенных вершин
    queue<int> q;// Стандартная очередь
    //Включаем таймер
    auto start_time = high_resolution_clock::now();
    //Начало обхода со стартовой вершины
    visited[start] = true;
    q.push(start);
    //Пока очередь не пуста
    while (!q.empty()) {
        int current = q.front();// Берем первую вершину из
очереди
        q.pop();// Удаляем ее из очереди
        result.push_back(current);// Добавляем в результат
        //Просматриваем всех соседей текущей вершины через
матрицу смежности
        for (int i = 0; i < V; i++) {
            //Если есть ребро и вершина еще не посещена
            if (adjMatrix[current][i] == 1 && !visited[i]) {
                visited[i] = true;// Помечаем как посещенную
                q.push(i);// Добавляем в очередь для дальнейшего
обхода
            }
        }
    }
    //Останавливаем таймер
    auto end_time = high_resolution_clock::now();
    auto duration = duration_cast<microseconds>(end_time -
start_time);
    cout << "Время обхода (матрица + queue): " <<
duration.count() << " микросекунд" << endl;
    return result;
}

```

```

//Обход в ширину с использованием списков смежности и
стандартной очереди (Задание 1*)
vector<int> BFS_List_StdQueue(int start) {
    vector<int> result;// Вектор для хранения порядка обхода
    vector<bool> visited(V, false);// Вектор посещенных вершин
    queue<int> q;// Стандартная очередь
    //Включаем таймер
    auto start_time = high_resolution_clock::now();
    //Начало обхода со стартовой вершины
    visited[start] = true;
    q.push(start);
    //Основной цикл
    while (!q.empty()) {
        int current = q.front();// Берем первую вершину из
очереди
        q.pop();// Удаляем ее из очереди
        result.push_back(current);// Добавляем в результат
        //Просматриваем всех соседей через список смежности
        AdjListNode* neighbor = adjLists[current].head;

```

```

while (neighbor != nullptr) {
    //Если соседняя вершина еще не посещена
    if (!visited[neighbor->dest]) {
        visited[neighbor->dest] = true; // Помечаем как
посещенную
        q.push(neighbor->dest); // Добавляем в очередь
    }
    neighbor = neighbor->next; // Переход к следующему
соседу
}
}
//Останавливаем таймер
auto end_time = high_resolution_clock::now();
auto duration = duration_cast<microseconds>(end_time -
start_time);
cout << "Время обхода (списки + queue): " <<
duration.count() << " микросекунд" << endl;
return result;
}

//Обход в ширину с использованием матрицы смежности и
самодельной очереди
vector<int> BFS_Matrix_CustomQueue(int start) {
    vector<int> result; // Вектор для хранения порядка обхода
    vector<bool> visited(V, false); // Вектор посещенных вершин
    CustomQueue q; // Самодельная очередь
    //Включаем таймер
    auto start_time = high_resolution_clock::now();
    //Начало обхода со стартовой вершины
    visited[start] = true;
    q.push(start);
    while (!q.empty()) {
        int current = q.frontValue(); // Берем первую вершину из
очереди
        q.pop(); // Удаляем ее из очереди
        result.push_back(current); // Добавляем в результат
        //Просматриваем всех соседей текущей вершины через
матрицу смежности
        for (int i = 0; i < V; i++) {
            //Если есть ребро и вершина еще не посещена
            if (adjMatrix[current][i] == 1 && !visited[i]) {
                visited[i] = true; // Помечаем как посещенную
                q.push(i); // Добавляем в самодельную очередь
            }
        }
    }
    //Останавливаем таймер
    auto end_time = high_resolution_clock::now();
    auto duration = duration_cast<microseconds>(end_time -
start_time);
    cout << "Время обхода (матрица + custom queue): " <<
duration.count() << " микросекунд" << endl;
    return result;
}

```



```

//Обход в ширину с использованием списков смежности и
самодельной очереди
vector<int> BFS_List_CustomQueue(int start) {
    vector<int> result;// Вектор для хранения порядка обхода
    vector<bool> visited(V, false);// Вектор посещенных вершин
    CustomQueue q;// Самодельная очередь
    //Включаем таймер
    auto start_time = high_resolution_clock::now();
    //Начало обхода со стартовой вершины
    visited[start] = true;
    q.push(start);
    while (!q.empty()) {
        int current = q.frontValue();// Берем первую вершину из
очереди
        q.pop();// Удаляем ее из очереди
        result.push_back(current);// Добавляем в результат
        //Просматриваем всех соседей через список смежности
        AdjListNode* neighbor = adjLists[current].head;
        while (neighbor != nullptr) {
            //Если соседняя вершина еще не посещена
            if (!visited[neighbor->dest]) {
                visited[neighbor->dest] = true;// Помечаем как
посещенную
                q.push(neighbor->dest);// Добавляем в
самодельную очередь
            }
            neighbor = neighbor->next;// Переход к следующему
соседу
        }
    }
    //Останавливаем таймер
    auto end_time = high_resolution_clock::now();
    auto duration = duration_cast<microseconds>(end_time -
start_time);
    cout << "Время обхода (списки + custom queue): " <<
duration.count() << " микросекунд" << endl;
    return result;
}
//Вывод результатов
static void printBFSResult(const vector<int>& result, const
string& method) {
    cout << "Результат обхода в ширину (" << method << "): ";
    for (size_t i = 0; i < result.size(); i++) {
        cout << result[i];
        if (i != result.size() - 1) {
            cout << " -> ";
        }
    }
    cout << endl << endl;
}
//Геттер для получения количества вершин
int getVertexCount() const {
    return V;
}
};

```

```

//Функция для ввода целых чисел
int safeInputInt(const string& prompt, int minVal =
numeric_limits<int>::min(), int maxVal = numeric_limits<int>::max())
{
    int value;
    while (true) {
        cout << prompt;
        cin >> value;
        //Проверка на некорректный ввод
        if (cin.fail()) {
            cin.clear();// Сбрасываем флаг ошибки
            cin.ignore(numeric_limits<streamsize>::max(), '\n');//
Очищаем буфер ввода
            cout << "Ошибка: введите корректное число!" << endl;
        }
        //Проверка на выход за границы диапазона
        else if (value < minVal || value > maxVal) {
            cout << "Ошибка: число должно быть в диапазоне " <<
minVal << "-" << maxVal << "!" << endl;
            cin.ignore(numeric_limits<streamsize>::max(), '\n');
        }
        else {
            //Успешный ввод - очищаем буфер и возвращаем значение
            cin.ignore(numeric_limits<streamsize>::max(), '\n');
            return value;
        }
    }
}

//Менюшка
void displayMainMenu() {
    cout << "\nМеню" << endl;
    cout << "1. Показать текущий граф" << endl;
    cout << "2. Выполнить обход графа" << endl;
    cout << "3. Изменить стартовую вершину" << endl;
    cout << "4. Регенерировать граф" << endl;
    cout << "5. Создать новый граф" << endl;
    cout << "0. Выход" << endl;
}

//Менюшка методов обходов
void displayBFSMenu() {
    cout << "\nВыберите метод обхода" << endl;
    cout << "1. Матрица смежности + стандартная очередь" << endl;
    cout << "2. Списки смежности + стандартная очередь" << endl;
    cout << "3. Матрица смежности + самодельная очередь" << endl;
    cout << "4. Списки смежности + самодельная очередь" << endl;
    cout << "5. Все методы последовательно" << endl;
    cout << "0. Назад в главное меню" << endl;
}

int main() {
    setlocale(LC_ALL, "Russian");
    int V = 0;// Количество вершин
    int start = 0;// Стартовая вершина для обхода
    Graph* g = nullptr;// Указатель на граф (изначально не создан)

```

```

//Первоначальный ввод параметров графа
V = safeInputInt("Введите количество вершин графа: ", 1);
start = safeInputInt("Введите стартовую вершину (0-" +
to_string(V - 1) + "): ", 0, V - 1);
//Создание и генерация начального графа
g = new Graph(V);
g->generateRandomGraph();
int mainChoice;
do {
    displayMainMenu();
    mainChoice = safeInputInt("Ваш выбор: ", 0, 5);
    vector<int> result;// Для хранения результатов обхода
    switch (mainChoice) {
        case 1:
            if (g != nullptr) {
                cout << "Количество вершин: " << V << endl;
                cout << "Стартовая вершина: " << start << endl;
                g->printAdjMatrix();
                g->printAdjLists();
            }
            else {
                cout << "Граф не создан!" << endl;
            }
            break;

        case 2:
            if (g != nullptr) {
                int bfsChoice;
                do {
                    displayBFSMenu();
                    bfsChoice = safeInputInt("Ваш выбор: ", 0, 5);
                    switch (bfsChoice) {
                        case 1:
                            cout << "\nМетод 1: Матрица смежности +
стандартная очередь" << endl;
                            result = g->BFS_Matrix_StdQueue(start);
                            Graph::printBFSResult(result, "матрица +
queue");
                            break;

                        case 2:
                            cout << "\nМетод 2: Списки смежности +
стандартная очередь" << endl;
                            result = g->BFS_List_StdQueue(start);
                            Graph::printBFSResult(result, "списки +
std::queue");
                            break;

                        case 3:
                            cout << "\nМетод 3: Матрица смежности +
самодельная очередь" << endl;
                            result = g->BFS_Matrix_CustomQueue(start);
                            Graph::printBFSResult(result, "матрица +
custom queue");
                            break;
                    }
                }
            }
    }
} while (mainChoice != 0);

```

```

        case 4:
            cout << "\nМетод 4: Списки смежности +
самодельная очередь" << endl;
            result = g->BFS_List_CustomQueue(start);
            Graph::printBFSResult(result, "списки +
custom queue");
            break;

        case 5:
            cout << "\nВсе методы последовательно" <<
endl;
            cout << "\n1. Матрица смежности +
стандартная очередь:" << endl;
            result = g->BFS_Matrix_StdQueue(start);
            Graph::printBFSResult(result, "матрица +
queue");
            cout << "2. Списки смежности + стандартная
очередь:" << endl;
            result = g->BFS_List_StdQueue(start);
            Graph::printBFSResult(result, "списки +
queue");
            cout << "3. Матрица смежности + самодельная
очередь:" << endl;
            result = g->BFS_Matrix_CustomQueue(start);
            Graph::printBFSResult(result, "матрица +
custom queue");
            cout << "4. Списки смежности + самодельная
очередь:" << endl;
            result = g->BFS_List_CustomQueue(start);
            Graph::printBFSResult(result, "списки +
custom queue");
            break;

        case 0:
            break;
    }
    } while (bfsChoice != 0);
}
else {
    cout << "Граф не создан!" << endl;
}
break;

case 3:
    if (g != nullptr) {
        start = safeInputInt("Введите новую стартовую
вершину (0-" + to_string(V - 1) + "): ", 0, V - 1);
        cout << "Стартовая вершина изменена на: " << start
<< endl;
    }
    else {
        cout << "Граф не создан!" << endl;
    }
    break;

case 4:

```

```

        if (g != nullptr) {
            g->generateRandomGraph();
            g->printAdjMatrix();
            g->printAdjLists();
        }
        else {
            cout << "Граф не создан!" << endl;
        }
        break;

    case 5:
    {
        int newV = safeInputInt("Введите количество вершин для
нового графа: ", 1);
        V = newV;
        delete g;
        g = new Graph(V);
        g->generateRandomGraph();
        start = safeInputInt("Введите стартовую вершину (0-" +
to_string(V - 1) + "): ", 0, V - 1);
        g->printAdjMatrix();
        g->printAdjLists();
    }
    break;

    case 0:
        break;
    }

} while (mainChoice != 0);
//Освобождение памяти перед выходом
if (g != nullptr) {
    delete g;
}
return 0;
}

```

Текст к программе

Данная программа реализует алгоритм обхода графа в ширину (BFS) с возможностью сравнения эффективности различных структур данных. Программа генерирует случайный неориентированный граф и предоставляет инструменты для его визуализации и анализа. Основной функционал включает отображение графа в виде матрицы смежности и списков смежности, выполнение обхода в ширину с использованием стандартной и самодельной очереди, а также замер времени выполнения для каждого метода. Пользователь может изменять параметры графа, выбирать стартовую вершину и сравнивать производительность разных подходов. Интерфейс программы организован в виде интерактивного меню с защитой от некорректного ввода. Реализация демонстрирует принципы работы с графами, организацию очередей и сравнительный анализ алгоритмов на C++.

Результаты работы программы

Результаты работы программы показаны на рисунках 2-3.

```

Введите количество вершин графа: 5
Введите стартовую вершину (0-4): 0

Меню
1. Показать текущий граф
2. Выполнить обход графа
3. Изменить стартовую вершину
4. Регенерировать граф
5. Создать новый граф
0. Выход
Ваш выбор: 1
Количество вершин: 5
Стартовая вершина: 0
Матрица смежности:
    0  1  2  3  4
0  0  0  0  1  0
1  0  0  1  1  1
2  0  1  0  1  0
3  1  1  1  0  1
4  0  1  0  1  0

Списки смежности:
0: 3
1: 4 -> 3 -> 2
2: 3 -> 1
3: 4 -> 2 -> 1 -> 0
4: 3 -> 1
  
```

Рисунок 1 – Создание графа с 5 вершинами(стартовая позиция 0)

```

Меню
1. Показать текущий граф
2. Выполнить обход графа
3. Изменить стартовую вершину
4. Регенерировать граф
5. Создать новый граф
0. Выход
Ваш выбор: 2

Выберите метод обхода
1. Матрица смежности + стандартная очередь
2. Списки смежности + стандартная очередь
3. Матрица смежности + самодельная очередь
4. Все методы последовательно
0. Назад в главное меню
Ваш выбор: 4

Все методы последовательно

1. Матрица смежности + стандартная очередь:
Время обхода (матрица + queue): 64 микросекунд
Результат обхода в ширину (матрица + queue): 0 -> 3 -> 1 -> 2 -> 4

2. Списки смежности + стандартная очередь:
Время обхода (списки + queue): 34 микросекунд
Результат обхода в ширину (списки + queue): 0 -> 3 -> 4 -> 2 -> 1

3. Матрица смежности + самодельная очередь:
Время обхода (матрица + custom queue): 30 микросекунд
Результат обхода в ширину (матрица + custom queue): 0 -> 3 -> 1 -> 2 -> 4
  
```

Рисунок 2 – Обход графа

```

Меню
1. Показать текущий граф
2. Выполнить обход графа
3. Изменить стартовую вершину
4. Регенерировать граф
5. Создать новый граф
0. Выход
Ваш выбор: 3
Введите новую стартовую вершину (0-4): 3
Стартовая вершина изменена на: 3

Меню
1. Показать текущий граф
2. Выполнить обход графа
3. Изменить стартовую вершину
4. Регенерировать граф
5. Создать новый граф
0. Выход
Ваш выбор: 2

Выберите метод обхода
1. Матрица смежности + стандартная очередь
2. Списки смежности + стандартная очередь
3. Матрица смежности + самодельная очередь
4. Все методы последовательно
0. Назад в главное меню
Ваш выбор: 4

Все методы последовательно

1. Матрица смежности + стандартная очередь:
Время обхода (матрица + queue): 34 микросекунд
Результат обхода в ширину (матрица + queue): 3 -> 0 -> 1 -> 2 -> 4

2. Списки смежности + стандартная очередь:
Время обхода (списки + queue): 70 микросекунд
Результат обхода в ширину (списки + queue): 3 -> 4 -> 2 -> 1 -> 0

3. Матрица смежности + самодельная очередь:
Время обхода (матрица + custom queue): 40 микросекунд
Результат обхода в ширину (матрица + custom queue): 3 -> 0 -> 1 -> 2 -> 4

```

Рисунок 4 – Изменение стартовой позиции

```

Меню
1. Показать текущий граф
2. Выполнить обход графа
3. Изменить стартовую вершину
4. Регенерировать граф
5. Создать новый граф
0. Выход
Ваш выбор: 4
Матрица смежности:
    0  1  2  3  4
0  0  1  0  0  0
1  1  0  1  0  1
2  0  1  0  1  1
3  0  0  1  0  0
4  0  1  1  0  0

Списки смежности:
0: 1
1: 4 -> 2 -> 0
2: 4 -> 3 -> 1
3: 2
4: 2 -> 1

```

Рисунок 5 – Регенерация графа

```

Меню
1. Показать текущий граф
2. Выполнить обход графа
3. Изменить стартовую вершину
4. Регенерировать граф
5. Создать новый граф
0. Выход
Ваш выбор: 5
Введите количество вершин для нового графа: 7
Введите стартовую вершину (0-6): 1
Матрица смежности:
    0  1  2  3  4  5  6
0  0  1  1  0  1  0  0
1  1  0  0  0  1  0  0
2  1  0  0  0  0  0  1
3  0  0  0  0  1  0  0
4  1  1  0  1  0  1  0
5  0  0  0  0  1  0  1
6  0  0  1  0  0  1  0

Списки смежности:
0: 4 -> 2 -> 1
1: 4 -> 0
2: 6 -> 0
3: 4
4: 5 -> 3 -> 1 -> 0
5: 6 -> 4
6: 5 -> 2

```

Рисунок 6 – Создание совершенно нового графа


```

Меню
1. Показать текущий граф
2. Выполнить обход графа
3. Изменить стартовую вершину
4. Регенерировать граф
5. Создать новый граф
0. Выход
Ваш выбор: -5
Ошибка: число должно быть в диапазоне 0-5!

```

Рисунок 7 – Попытка ввода некорректного значения

```

Введите количество вершин графа: 10
Введите стартовую вершину (0-9): 0

Меню
1. Показать текущий граф
2. Выполнить обход графа
3. Изменить стартовую вершину
4. Регенерировать граф
5. Создать новый граф
0. Выход
Ваш выбор: 2

Выберите метод обхода
1. Матрица смежности + стандартная очередь
2. Списки смежности + стандартная очередь
3. Матрица смежности + самодельная очередь
4. Списки смежности + самодельная очередь
5. Все методы последовательно
0. Назад в главное меню
Ваш выбор: 1

Метод 1: Матрица смежности + стандартная очередь
Время обхода (матрица + queue): 52 микросекунд
Результат обхода в ширину (матрица + queue): 0 -> 4 -> 5 -> 6 -> 2 -> 3 -> 7 -> 9 -> 8 -> 1

Выберите метод обхода
1. Матрица смежности + стандартная очередь
2. Списки смежности + стандартная очередь
3. Матрица смежности + самодельная очередь
4. Списки смежности + самодельная очередь
5. Все методы последовательно
0. Назад в главное меню
Ваш выбор: 3

Метод 3: Матрица смежности + самодельная очередь
Время обхода (матрица + custom queue): 44 микросекунд
Результат обхода в ширину (матрица + custom queue): 0 -> 4 -> 5 -> 6 -> 2 -> 3 -> 7 -> 9 -> 8 -> 1

```

Рисунок 8 – Скорость обхода графа с 10 вершинами 2 методами


```

Введите количество вершин графа: 3000
Введите стартовую вершину (0-2999): 0

Меню
1. Показать текущий граф
2. Выполнить обход графа
3. Изменить стартовую вершину
4. Регенерировать граф
5. Создать новый граф
0. Выход
Ваш выбор: 2

Выберите метод обхода
1. Матрица смежности + стандартная очередь
2. Списки смежности + стандартная очередь
3. Матрица смежности + самодельная очередь
4. Списки смежности + самодельная очередь
5. Все методы последовательно
0. Назад в главное меню
Ваш выбор: 1

Метод 1: Матрица смежности + стандартная очередь
Время обхода (матрица + queue): 1569593 микросекунд

```

Рисунок 11 – Скорость обхода графа с 3000 вершинами 2 методами (Библиотека)

```

Выберите метод обхода
1. Матрица смежности + стандартная очередь
2. Списки смежности + стандартная очередь
3. Матрица смежности + самодельная очередь
4. Списки смежности + самодельная очередь
5. Все методы последовательно
0. Назад в главное меню
Ваш выбор: 3

Метод 3: Матрица смежности + самодельная очередь
Время обхода (матрица + custom queue): 1551368 микросекунд

```

Рисунок 11 – Скорость обхода графа с 3000 вершинами 2 методами (Самодельный)

Выводы

В ходе выполнения лабораторной работы были успешно реализованы и протестированы различные подходы к обходу графов в ширину с использованием различных структур данных. Разработанная программа включает четыре основные реализации алгоритма BFS: обход с матрицей смежности и стандартной очередью, обход со списками смежности и стандартной очередью, обход с матрицей смежности и самодельной очередью, а также обход со списками смежности и самодельной очередью. Особенностью

реализации является генерация случайных неориентированных графов с возможностью динамического изменения параметров и стартовой вершины обхода. Программа обеспечивает корректный обход всех вершин связной компоненты графа с измерением времени выполнения для каждого метода. Интерфейс предоставляет удобное меню для навигации между различными режимами работы, включая просмотр текущего графа, выполнение обходов, изменение параметров и регенерацию графа. В процессе тестирования подтверждена корректность работы всех реализаций алгоритма BFS. Все методы показывают идентичные результаты обхода при использовании одинаковых входных данных, что свидетельствует о правильной реализации алгоритмов и их устойчивой работе при различных конфигурациях графов.